

PDF Vectorisation Pipeline

Assessment - LangChain / LangGraph Implementation

Language	Python (required)
Framework	LangChain / LangGraph

OVERVIEW

Build a pipeline with two components using LangChain: (1) an ingestion script that loads a PDF using a LangChain document loader, splits it with a LangChain text splitter, generates embeddings via a LangChain embedding integration, and persists them to a LangChain-compatible vector store; and (2) a retrieval endpoint that queries the vector store directly using similarity search. There is no LLM call on the retrieval side and no QA or conversational chain. Return raw chunk results only.

PART 1 - INGESTION

Write a script using LangChain primitives that performs the following steps in order:

1. Load the PDF using a LangChain document loader (e.g. PyPDFLoader, PDFMinerLoader, UnstructuredPDFLoader).
2. Split the loaded document using a LangChain text splitter (e.g. RecursiveCharacterTextSplitter). Choose chunk size and overlap thoughtfully - justify both in the README.
3. Generate embeddings using a LangChain embedding integration (e.g. OpenAIEmbeddings, HuggingFaceEmbeddings, CohereEmbeddings).
4. Persist the embeddings to a LangChain-compatible vector store (e.g. Chroma, FAISS, Pinecone). Use the vector store's built-in persistence method.

The script must be runnable as:

```
python ingest.py --file document.pdf
```

LangGraph (optional, rewarded): If you use LangGraph, model the ingestion as a graph with clearly named nodes - for example: load → split → embed → store. Include a printed node list or a diagram in your README.

PART 2 - RETRIEVAL ENDPOINT

Expose a single HTTP endpoint. Embed the incoming query using the same model from ingestion and call similarity_search_with_score() (or the equivalent method) on the vector store. Return the top-k results directly.

Method	Endpoint	Description
POST	/query	Request body: { "query": string, "top_k": int }. Response: array of { chunk_text, page_number, score }.

Do not wrap retrieval in a RetrievalQA chain, ConversationalRetrievalChain, or any LLM chain. The endpoint must call the vector store's similarity search method directly and return raw results.

CONSTRAINTS

- Python is required. LangChain core must be used for loading, splitting, embedding, and vector store operations.
- LangGraph is optional but will be evaluated positively if the graph is well-structured and documented.
- Do not use RetrievalQA, ConversationalRetrievalChain, or any pre-built RAG chain for the retrieval endpoint.
- The pipeline must run successfully on the sample PDF provided on the assessment platform.

DELIVERABLES

- Ingestion script and retrieval server, fully runnable from your README instructions.
- README: setup and install steps (include a requirements.txt), justification for your choice of loader, splitter, embedding model, and vector store, and sample output from the provided PDF.
- results.json: output of at least three different queries run against the provided PDF, in the response format above.
- If LangGraph is used: a printed node list or diagram showing the ingestion graph structure, included in the README.

EVALUATION

Criterion	Weight	What we look for
Pipeline runs end-to-end without errors	35%	Ingestion completes; retrieval returns relevant results.
Correct use of LangChain primitives for each stage	25%	Right component used for the right purpose; no misuse of chains.
Retrieval quality - relevance of returned chunks	20%	results.json entries are semantically relevant to each query.
README quality and component choices with reasoning	15%	Can justify each LangChain component selected.
LangGraph graph structure and documentation (bonus)	+5%	Named nodes, documented flow, inspectable graph.

Instructions

Kindly create a private GitHub repository named: {yourName_Nestack_Submission} (e.g., roy_Nestack_Submission) and add the following contributors for evaluation:

- bishal@nestack.com
- sannidhya@nestack.com
- sanjay@nestack.com

Your submission must include:

- A ZIP file of your complete codebase
- A live deployment link
- Any necessary credentials if your project includes authentication

Additionally, ensure your repository includes a README.md with:

- The deployment link clearly mentioned
- Setup and run instructions if applicable

NOTE: PUBLIC REPOSITORIES ARE DISQUALIFIED FROM THE HIRING PROCESS